

ARIA · SECURITY REVIEW

Aria

ERC 20 Native and Token Review.

A security review of ERC 20 Native and Token Review.

CLIENT

Aria

PROJECT

ERC 20 Native and
Token Review

AUDITORS

Guardian

AUDIT WINDOW

October 29 –
November 5, 2025

FINAL REPORT

November 12, 2025

§ 00

Contents.

51 pages — overview, confidence ranking, scope, 40 findings, disclaimer, and about Guardian.

§ 01	Overview	P. 03
§ 02	Confidence Ranking	P. 04
§ 03	Scope & Classifications	P. 05
	Files in Scope	P. 06
§ 04	Findings	P. 07
	Main Review · 33 findings	P. 10
	Remediation Review · 7 findings	P. 43
§ 05	Disclaimer	P. 50
§ 06	About Guardian	P. 51

§ 01

Overview.

Guardian reviewed ERC 20 Native and Token Review across 2 review rounds. The final report includes 1 Medium, 21 Low, 18 Informational findings.

AUDIT FIRM	REPOSITORY
Guardian	AriaProtocol/main-contracts
CLIENT	METHODOLOGY
Aria	Smart contract · Manual · Main Review · Remediation Review
PROJECT	MAIN REVIEW COMMIT
ERC 20 Native and Token Review	1daed28ea7ba
PREPARED BY	
Guardian	
AUDIT WINDOW	
October 29 – November 5, 2025	
FINAL REPORT	
November 12, 2025	

1.1 Confidence Ranking

5 / 5

Very High Confidence

Guardian assigns a Confidence Ranking of 5 out of 5, combining finding severity, remediation outcome, and the reviewer’s assessment of latent risk. The full scale is on the next page.

1	2	3	4	5
VERY LOW CONFIDENCE	LOW CONFIDENCE	MODERATE CONFIDENCE	HIGH CONFIDENCE	VERY HIGH CONFIDENCE

1.2 Findings at a Glance

SEVERITY	TOTAL	PENDING	ACK	PARTIAL	RESOLVED
Critical	0	0	0	0	0
High	0	0	0	0	0
Medium	1	0	0	0	1
Low	21	0	2	2	17
Informational	18	0	1	0	17
All severities	40	0	3	2	35

§ 02

Confidence Ranking.

Guardian assigns a project-level confidence score using the scale below. The ranking combines finding severity, remediation outcome, and the reviewer's assessment of latent risk.

CONFIDENCE RANKING	DEFINITION AND RECOMMENDATION	RISK PROFILE
5 Very High Confidence ASSIGNED	Codebase is mature, clean, and secure. No High or Critical vulnerabilities were found. Follows modern best practices with high test coverage and thoughtful design. Recommendation: code is highly secure at time of audit.	0 High/Critical findings and few Low/Medium severity findings.
4 High Confidence	Code is clean, well-structured, and adheres to best practices. Design patterns are sound and test coverage is strong. Recommendation: suitable for deployment after remediations; consider periodic review with changes.	0-1 High/Critical findings per engagement week and little to no Medium severity findings.
3 Moderate Confidence	Code is functional but shows gaps in defensive programming or test coverage. Recommendation: remediate the findings and re-review the changes before deployment.	Some High/Critical findings, with various Medium severity findings.
2 Low Confidence	Recurring issues suggest systemic gaps rather than isolated mistakes. Recommendation: remediate, then commission a follow-up review at a frozen commit.	Over 6 High and Critical findings across the engagement.
1 Very Low Confidence	The volume and severity of findings indicate the codebase is not ready for production. Recommendation: substantial rework, then a full re-review.	Over 12 High and Critical findings across the engagement.

§ 03

Scope & Classifications.

Guardian reviewed ERC 20 Native and Token Review in `AriaProtocol/main-contracts`. The scope includes 323 source lines across 7 files.

3.1 Severity Matrix

	IMP · HIGH	IMP · MED	IMP · LOW
LIK · HIGH	Critical	High	Medium
LIK · MED	High	Medium	Low
LIK · LOW	Medium	Low	Low

IMPACT · HIGH

Significant loss of assets, harm to a group of users, or core protocol functionality is disrupted.

IMPACT · MEDIUM

Small amount of funds can be lost, or ancillary functionality is affected. Users may experience reduced or delayed receipt.

IMPACT · LOW

Unexpected behavior in some functionality that is notable but does not meet the criteria for higher severity.

LIKELIHOOD · HIGH

Possible with reasonable assumptions that mimic on-chain conditions; cost is low compared to amount gained.

LIKELIHOOD · MEDIUM

Possible only in uncommon cases or requires a large amount of capital relative to disruption caused.

LIKELIHOOD · LOW

Unlikely to ever occur in production.

Files in Scope.

FILE	NSLOC	TOTAL
contracts/claim/erc20-native/ERC20NativeClaim.sol	24	45
contracts/claim/erc20-native/vesting/BaseVesting.sol	71	141
contracts/claim/erc20-native/vesting/CliffReleaseVesting.sol	73	142
contracts/claim/erc20-native/vesting/SnapshotVesting.sol	32	58
contracts/claim/erc20-native/shared/Claim.sol	14	23
contracts/claim/erc20-native/shared/ClaimAdmin.sol	95	142
contracts/claim/erc20-native/shared/MerkleWhitelist.sol	14	18

§ 04

Findings.

Grouped by round. Visible IDs reset by severity within each round, so a finding is named the same way in the report as it was in the channel it was written in.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
MAIN REVIEW					
M-01	Token Configuration DoS	DoS	MEDIUM	RESOLVED	p. 10
L-01	Invalid ClaimAdmin__ZeroAddress Revert	Best Practices	LOW	RESOLVED	p. 11
L-02	Underflow When Checking Releasable	Warning	LOW	RESOLVED	p. 12
L-02.2	OZ Version Warning	Warning	LOW	RESOLVED	p. 13
L-03.2	Lack Of Address(0) Validation On Owner	Validation	LOW	RESOLVED	p. 14
L-03	Fee-on-transfer And Rebase Tokens Not Supported	Warning	LOW	RESOLVED	p. 15
L-04	Zero Claims Allowed	Events	LOW	RESOLVED	p. 16
L-05	Missing whenNotPaused Modifiers	Validation	LOW	ACKNOWLEDGED	p. 17
L-06	Reliance On Contract Balance Risks	Warning	LOW	ACKNOWLEDGED	p. 18
L-07	Start and End Time Movement DoS Claims	DoS	LOW	PARTIALLY RESOLVED	p. 19
L-08	Cliff Release Percentage Change Leads To DoS	DoS	LOW	RESOLVED	p. 20
L-09	Allocation Update Leads To DoS	DoS	LOW	RESOLVED	p. 21
L-10	Updating Start Time Will DoS	DoS	LOW	RESOLVED	p. 22
L-11	Linear Duration Can Underflow	DoS	LOW	RESOLVED	p. 23

Findings continued.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
L-12	DoS On Merkle Root Change	DoS	LOW	RESOLVED	p. 24
L-13	Native Claims To Smart Contract May Fail	Warning	LOW	PARTIALLY RESOLVED	p. 25
L-14	depositedRewards Unused	Documentation	LOW	RESOLVED	p. 26
L-15	Total Allocated Not Used For Claim Calculation	Warning	LOW	RESOLVED	p. 27
L-16	Snapshot Leaves Must Be Consistent	Warning	LOW	RESOLVED	p. 28
L-17	Grant/Revoke Role Always Emits Event	Events	LOW	RESOLVED	p. 29
I-01	Unecessary Calculation	Gas Optimization	INFORMATIONAL	RESOLVED	p. 30
I-01.2	Unused ERC20Capped Import	Best Practices	INFORMATIONAL	RESOLVED	p. 31
I-02	Temporary Negative Allocation Reverts	Warning	INFORMATIONAL	RESOLVED	p. 32
I-02.2	Arbitrary Users Can Burn	Documentation	INFORMATIONAL	RESOLVED	p. 33
I-03.2	DEFAULT ADMIN Used For Minting	Warning	INFORMATIONAL	RESOLVED	p. 34
I-04.2	Total Supply Less Than Minted	Documentation	INFORMATIONAL	RESOLVED	p. 35
I-05.2	Consider Zero Amount Check	Validation	INFORMATIONAL	ACKNOWLEDGED	p. 36
I-03	Lack Of Zero Address Check On Admin	Validation	INFORMATIONAL	RESOLVED	p. 37
I-04	Unused Import	Superfluous Code	INFORMATIONAL	RESOLVED	p. 38
I-05	Allocations Made For Zero Address	Warning	INFORMATIONAL	RESOLVED	p. 39
I-06	100% Vesting At Cliff Not Possible	Best Practices	INFORMATIONAL	RESOLVED	p. 40
I-07	Potential Overflow On Released Calculation	Warning	INFORMATIONAL	RESOLVED	p. 41

Findings continued.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
I-08	Streamlining Storage	Warning	INFORMATIONAL	RESOLVED	p. 42

REMEDIATION REVIEW

L-01	Invalid ClaimAdmin__ZeroAddress Revert	Best Practices	LOW	RESOLVED	p. 43
L-02	Arbitrary Token Can Be Deposited	Validation	LOW	RESOLVED	p. 44
I-01	startTime After cliff Is Possible	Validation	INFORMATIONAL	RESOLVED	p. 45
I-02	PausableUpgradable Not Used	Best Practices	INFORMATIONAL	RESOLVED	p. 46
I-03	Small Amounts Rounds Down To 0	Warning	INFORMATIONAL	RESOLVED	p. 47
I-04	Redundant Check	Best Practices	INFORMATIONAL	RESOLVED	p. 48
I-05	Invalid Comment For Native Token	Documentation	INFORMATIONAL	RESOLVED	p. 49

MEDIUM **M-01**

RESOLVED

Token Configuration DoS

CATEGORY DoS
LOCATION SnapshotVesting.sol
REVIEW Main Review

DESCRIPTION

The token defaults to address(0) (native). While token remains unset, anyone can call deposit() to fund the contract with ETH and, if whitelisted in the merkle tree and within the window, call claim() to withdraw ETH to themselves. Because setToken() is permanently disabled after totalClaimed > 0, a single successful ETH claim irreversibly locks the distribution to native token, preventing the admin from switching to the intended ERC20. An attacker can front-run by depositing ETH and claiming as soon as the window opens if the admin forgets to set the token first.

RECOMMENDATION

Require explicit token configuration before `startTime` or use a sentinel value instead of `address(0)`.

RESOLUTION

Status: Resolved.

<https://github.com/AriaProtocol/main-contracts/pull/149>

LOW **L-01**

RESOLVED

Invalid ClaimAdmin__ZeroAddress Revert

CATEGORY	Best Practices
LOCATION	https://github.com/GuardianOrg/main-contracts-team1-1761523410273/blob/1daed28ea7ba4c30d8020e3948a07a3477ba4d3f/contracts/claim/erc20-native/vesting/BaseVesting.sol#L70
REVIEW	Main Review

— DESCRIPTION

When `token_ != token`, the functions `releasable` and `vestedAmount` revert with `ClaimAdmin__ZeroAddress()`, which does not represent the true reason for the error.

— RECOMMENDATION

Change the revert name to match the cause of the error.

— RESOLUTION

Status: Resolved.

<https://github.com/AriaProtocol/main-contracts/pull/150>

LOW **L-02**

RESOLVED

Underflow When Checking Releasable

CATEGORY	Warning
LOCATION	https://github.com/GuardianOrg/main-contracts-team1-1761523410273/blob/1daed28ea7ba4c30d8020e3948a07a3477ba4d3f/contracts/claim/erc20-native/vesting/BaseVesting.sol#L71
REVIEW	Main Review

DESCRIPTION

Function `releasable` risks underflow if an admin were to change the cliff with `setCliff` such that `vestedAmount(token_, uint64(block.timestamp))` becomes zero and `totalClaimed` is positive. This would block integrators from viewing the true amount of releasable funds.

RECOMMENDATION

Do not move the cliff after claims have been made.

RESOLUTION

Status: Resolved.

<https://github.com/AriaProtocol/main-contracts/pull/151>

LOW **L-02.2**

RESOLVED

OZ Version Warning

CATEGORY	Warning
LOCATION	Global
REVIEW	Main Review

— DESCRIPTION

Openzeppelin-contracts v5.3.0 are susceptible to an out-of-bounds read error in the Bytes lib:
<https://security.snyk.io/vuln/SNYK-JS-OPENZEPPELINCONTRACTS-10773735>

Although the token does not use this lib, it may be prudent to use the latest OZ version.

— RECOMMENDATION

Consider upgrading the `openzeppelin-contracts` version.

— RESOLUTION

Status: Resolved.

<https://github.com/AriaProtocol/main-contracts/pull/146>

LOW **L-03.2**

RESOLVED

Lack Of Address(0) Validation On Owner

CATEGORY Validation
LOCATION Aria.sol: 24
REVIEW Main Review

— DESCRIPTION

The Aria token's constructor does not have an explicit check to ensure the `initialOwner` is not the zero address. It may be prudent to add this validation to ensure safe deployment.

— RECOMMENDATION

Add zero-address check before granting the role.

— RESOLUTION

Status: Resolved.

<https://github.com/AriaProtocol/main-contracts/pull/146>

LOW **L-03**

RESOLVED

Fee-on-transfer And Rebase Tokens Not Supported

CATEGORY Warning
LOCATION Global
REVIEW Main Review

DESCRIPTION

Fee-on-transfer and rebasing tokens are not currently supported by the system. For example, `Claim._claim` delivers the exact `_amount`. If the configured token is fee-on-transfer / rebasing / otherwise non-standard, the hook still credits `claimed` and `totalClaimed` with the full `_amount` even though the user receives less (or more).

RECOMMENDATION

Restrict tokens to plain ERC-20/native asset.

RESOLUTION

Status: Resolved.

<https://github.com/AriaProtocol/main-contracts/pull/152>

LOW **L-04**

RESOLVED

Zero Claims Allowed

CATEGORY	Events
LOCATION	ERC20NativeClaim.sol
REVIEW	Main Review

DESCRIPTION

`ERC20NativeClaim.claim` allows a claim amount of zero, hence the event `Claimed` could be emitted repeatedly as `_claimChecks` only validates that `if (claimed[msg.sender] != 0)`.

RECOMMENDATION

Either block zero claim amounts or clearly document this behavior.

RESOLUTION

Status: Resolved.

<https://github.com/AriaProtocol/main-contracts/pull/153>

LOW **L-05**

ACKNOWLEDGED

Missing whenNotPaused Modifiers

CATEGORY Validation
LOCATION Global
REVIEW Main Review

DESCRIPTION

The `deposit` functions of the `ClaimAdmin` contract are callable by everyone but are not pausable.

RECOMMENDATION

Consider to add `whenNotPaused` to all external functions to follow best practices.

PROOF OF CONCEPT <https://github.com/GuardianOrg/main-contracts-team1-1761523143397/blob/1daed28ea7ba4c30d8020e3948a07a3477ba4d3f/contracts/claim/erc20-native/shared/ClaimAdmin.sol#L55-L73>

RESOLUTION

Status: Acknowledged.

LOW **L-06**

ACKNOWLEDGED

Reliance On Contract Balance Risks

CATEGORY	Warning
LOCATION	BaseVesting.sol
REVIEW	Main Review

— DESCRIPTION

If the admin withdraws tokens, the live balance shrinks. The recomputed `totalReleased` drops as well, and for users who already claimed based on the previous total allocation the new `userTotalReleased` can fall below their historical claimed amount, which makes `_toClaim` revert with `BaseVesting__AllocationDecreased`.

Also note that with direct contract donations or whenever contract balance exceeds the total allocated, the contract immediately treats the higher balance as though it had been part of the original allocation. The linear curve `_release` therefore jumps up, so every user can instantly claim their proportional share of the donation instead of the donation vesting over the remaining duration. Users will also pull more than their recorded allocation, and total claims will exceed total allocation/total snapshotted.

— RECOMMENDATION

Use a fixed ledger of "total allocated" tokens that is only ever updated by explicit funding events instead of reliance on `.balance` or `balanceOf`. Then `_toClaim` will not ignore the passed parameters.

— RESOLUTION

Status: Acknowledged.

LOW **L-07**

PARTIALLY RESOLVED

Start and End Time Movement DoS Claims

CATEGORY	DoS
LOCATION	Global
REVIEW	Main Review

DESCRIPTION

`setStartTime` and `setEndTime` remain callable by the admin after launch. Because every claim runs `_checkStartTime()` / `_checkEndTime()`, pushing `startTime` forward or pulling `endTime` back once users have begun claiming immediately bricks the flow for everyone else.

There is no guard comparable to `totalClaimed > 0`, so a routine timetable adjustment moves the eligible window outside the current block timestamp and all claim calls revert

RECOMMENDATION

Consider preventing start and end time updates after claims begun.

RESOLUTION

Status: Partially resolved.

<https://github.com/AriaProtocol/main-contracts/pull/154>

LOW **L-08**

RESOLVED

Cliff Release Percentage Change Leads To DoS

CATEGORY	DoS
LOCATION	https://github.com/GuardianOrg/main-contracts-team1-1761523410273/blob/1daed28ea7ba4c30d8020e3948a07a3477ba4d3f/contracts/claim/erc20-native/vesting/CliffReleaseVesting.sol#L91
REVIEW	Main Review

DESCRIPTION

The admin may lower `cliffReleasePercentage` at any time. After claims have started, the new curve in `_release` pushes the expected total release at the current timestamp below what has already been paid out. When `_toClaim` recomputes each user's, `userTotalReleased` becomes smaller than `claimed[msg.sender]` so every subsequent claim reverts with `BaseVesting__AllocationDecreased`. This permanently DoSes all claimants.

RECOMMENDATION

Forbid percentage decreases after `totalClaimed > 0`

RESOLUTION

Status: Resolved.

<https://github.com/AriaProtocol/main-contracts/pull/155>

LOW **L-09**

RESOLVED

Allocation Update Leads To DoS

CATEGORY	DoS
LOCATION	https://github.com/GuardianOrg/main-contracts-team1-1761523410273/blob/1daed28ea7ba4c30d8020e3948a07a3477ba4d3f/contracts/claim/erc20-native/vesting/CliffReleaseVesting.sol#L68
REVIEW	Main Review

DESCRIPTION

Updating allocations after any user has claimed can also DoS existing claimants. Function `updateAllocations` adjusts `totalAllocated`, which is later used as the denominator in `_toClaim`. If an admin onboards a new wallet or increases some other wallet once another user has already claimed, the recalculated share `userTotalReleased = userAmount * totalReleased / totalAmount` shrinks for the existing claimer, making it less than `claimed[msg.sender]` and triggering `BaseVesting__AllocationDecreased` on every future claim. In practice, a single late allocation can freeze all earlier participants.

Furthermore, any allocation decreases after other users have already claimed risk an ERC20InsufficientBalance revert since old `totalClaimed` history is maintained even if the allocations have changed or the admin withdrew tokens.

RECOMMENDATION

Disallow allocation changes once `totalClaimed > 0` or vesting begins.

RESOLUTION

Status: Resolved.

<https://github.com/AriaProtocol/main-contracts/pull/156>

LOW **L-10**

RESOLVED

Updating Start Time Will DoS

CATEGORY	DoS
LOCATION	https://github.com/GuardianOrg/main-contracts-team1-1761523410273/blob/1daed28ea7ba4c30d8020e3948a07a3477ba4d3f/contracts/claim/erc20-native/shared/ClaimAdmin.sol#L97
REVIEW	Main Review

— DESCRIPTION

Raising `startTime` once vesting has begun makes `_release` recompute with a later start, decreasing `totalReleased`. The next claim hits the same `AllocationDecreased` revert, freezing everyone until the original start time is restored.

— RECOMMENDATION

Prevent `startTime` from moving forward once vesting begins.

— RESOLUTION

Status: Resolved.

<https://github.com/AriaProtocol/main-contracts/pull/154>

LOW **L-11**

RESOLVED

Linear Duration Can Underflow

CATEGORY	DoS
LOCATION	https://github.com/GuardianOrg/main-contracts-team1-1761523410273/blob/1daed28ea7ba4c30d8020e3948a07a3477ba4d3f/contracts/claim/erc20-native/vesting/CliffReleaseVesting.sol#L115
REVIEW	Main Review

DESCRIPTION

If an admin shortens `endTime` below the current cliff, the subtraction in `linearDuration` underflows:

```
endTime - cliff;
```

Consequently, any `_release` function call reverts until the admin fixes the timestamps.

RECOMMENDATION

Modify `setEndTime` to ensure the new `endTime` doesn't fall below the current cliff.

RESOLUTION

Status: Resolved.

<https://github.com/AriaProtocol/main-contracts/pull/157>

LOW **L-12**

RESOLVED

DoS On Merkle Root Change

CATEGORY	DoS
LOCATION	https://github.com/GuardianOrg/main-contracts-team1-1761523410273/blob/1daed28ea7ba4c30d8020e3948a07a3477ba4d3f/contracts/iprwa/vault/admin/children/VaultWhitelistAdmin.sol#L28
REVIEW	Main Review

— DESCRIPTION

Updating the Merkle root after anyone has claimed reruns `_toClaim` with the new (snapshot,total) pair. If the refreshed leaf lowers either value, `userTotalReleased` drops below `claimed[msg.sender]`, so every subsequent claim reverts with `BaseVesting__AllocationDecreased`. This bricks all prior claimants even when the admin is just trying to admit new wallets or fix a typo in a snapshot amount.

— RECOMMENDATION

Lock the root once `totalClaimed > 0` or clearly document this risk.

— RESOLUTION

Status: Resolved.

<https://github.com/AriaProtocol/main-contracts/pull/158>

LOW **L-13**

PARTIALLY RESOLVED

Native Claims To Smart Contract May Fail

CATEGORY	Warning
LOCATION	Global
REVIEW	Main Review

— DESCRIPTION

Claims always transfer to msg.sender. If the whitelisted address is a smart contract without a payable receive/fallback, native token transfers will revert and the user cannot ever claim. There is no option to specify an alternative recipient address.

— RECOMMENDATION

Consider adding a claim variant to allow a different recipient. Otherwise, clearly document this behavior.

— RESOLUTION

Status: Partially resolved.

<https://github.com/AriaProtocol/main-contracts/pull/159>

LOW **L-14**

RESOLVED

depositedRewards Unused

CATEGORY	Documentation
LOCATION	ClaimAdmin.sol
REVIEW	Main Review

— DESCRIPTION

The `depositedRewards` which tracks deposits is never used and is not an accurate representation of how much funds are available for vesting. Also note that the passed `_token` does not have to match the token being vested out.

— RECOMMENDATION

Clearly document this behavior.

— RESOLUTION

Status: Resolved.

<https://github.com/AriaProtocol/main-contracts/pull/160>

LOW **L-15**

RESOLVED

Total Allocated Not Used For Claim Calculation

CATEGORY	Warning
LOCATION	Global
REVIEW	Main Review

— DESCRIPTION

The `_toClaim` function relies on `balance + totalClaimed` for calculation rather than `totalAllocated`. Consequently, users can pull more funds out than their intended allocation.

— RECOMMENDATION

Clearly document this is intended behavior.

— RESOLUTION

Status: Resolved.

<https://github.com/AriaProtocol/main-contracts/pull/161>

LOW **L-16**

RESOLVED

Snapshot Leaves Must Be Consistent

CATEGORY Warning
LOCATION SnapshotVesting.sol
REVIEW Main Review

DESCRIPTION

Each Merkle leaf encodes both the user's snapshotAmount and a caller-supplied `totalSnapshoted`. During claim, the contract takes that same pair and computes the payout as $\text{snapshotAmount} / \text{totalSnapshoted}$. If whoever prepares the snapshot accidentally (or intentionally) gives one address a smaller `totalSnapshoted` than everyone else, the proof will still verify and that address will receive a disproportionately large share of the vested tokens.

RECOMMENDATION

Ensure all leaves have the same `totalSnapshoted`.

RESOLUTION

Status: Resolved.

<https://github.com/AriaProtocol/main-contracts/pull/162>

LOW **L-17**

RESOLVED

Grant/Revoke Role Always Emits Event

CATEGORY	Events
LOCATION	AccessControlInternal.sol
REVIEW	Main Review

DESCRIPTION

Function `_grantRole` ignores the boolean returned by `EnumerableSet.add`, so re-granting a role that an account already holds still emits `RoleGranted`. Similarly, `_revokeRole` has the same issue on removal — a failed remove (account never had the role) still emits `RoleRevoked`. This may confuse off-chain indexers relying on events to reflect actual membership changes.

RECOMMENDATION

Clearly document this behavior.

RESOLUTION

Status: Resolved.

<https://github.com/AriaProtocol/main-contracts/pull/163>

INFORMATIONAL I-01

RESOLVED

Unnecessary Calculation

CATEGORY	Gas Optimization
LOCATION	https://github.com/GuardianOrg/main-contracts-team1-1761523410273/blob/1daed28ea7ba4c30d8020e3948a07a3477ba4d3f/contracts/claim/erc20-native/vesting/BaseVesting.sol#L114
REVIEW	Main Review

DESCRIPTION

In function `_vestingSchedule`, when `startTime` equals the timestamp, `_timePoints()` returns -1 (requires calculation) but `_release()` simply calculates: $(totalAllocation * 0) / duration() = 0$

This case could be shortcutted in `_timePoints()`.

RECOMMENDATION

If `timestamp <= startTime` return 0 timePoints.

RESOLUTION

Status: Resolved.

<https://github.com/AriaProtocol/main-contracts/pull/164>

INFORMATIONAL **I-01.2**

RESOLVED

Unused ERC20Capped Import

CATEGORY Best Practices
LOCATION Aria.sol: 6
REVIEW Main Review

DESCRIPTION

The Aria token contract imports `ERC20Capped` but does not use it. Instead, the Aria token implement its own capping logic.

RECOMMENDATION

Remove the extraneous import.

RESOLUTION

Status: Resolved.

<https://github.com/AriaProtocol/main-contracts/pull/147>

INFORMATIONAL I-02

RESOLVED

Temporary Negative Allocation Reverts

CATEGORY	Warning
LOCATION	https://github.com/GuardianOrg/main-contracts-team1-1761523410273/blob/1daed28ea7ba4c30d8020e3948a07a3477ba4d3f/contracts/claim/erc20-native/vesting/CliffReleaseVesting.sol#L81
REVIEW	Main Review

DESCRIPTION

The `updateAllocations()` function validates that each user's allocation is non-negative immediately after each iteration, rather than after all updates are processed. Consequently, multi-step allocation adjustments where a user's allocation temporarily goes negative before being corrected to be positive will revert.

RECOMMENDATION

Clearly document this behavior.

RESOLUTION

Status: Resolved.

<https://github.com/AriaProtocol/main-contracts/pull/165>

INFORMATIONAL **I-02.2**

RESOLVED

Arbitrary Users Can Burn

CATEGORY	Documentation
LOCATION	Aria.sol
REVIEW	Main Review

DESCRIPTION

The ERC20Burnable functions `burn` and `burnFrom` do not have access control, so any user holding the tokens can burn them. Ensure this intended behavior for the Aria token.

RECOMMENDATION

Ensure this intended behavior for the Aria token and clearly document this.

RESOLUTION

Status: Resolved.

<https://github.com/AriaProtocol/main-contracts/pull/147>

INFORMATIONAL **I-03.2**

RESOLVED

DEFAULT ADMIN Used For Minting

CATEGORY Warning
LOCATION Aria.sol
REVIEW Main Review

— DESCRIPTION

Currently the `DEFAULT_ADMIN_ROLE` performs minting operations. It may be prudent to have a separate `MINTER_ROLE` to handle operational tasks, similar to how AriaBadges is implemented.

— RECOMMENDATION

Consider using a separate role for minting, or clearly document this access control setup.

— RESOLUTION

Status: Resolved.

<https://github.com/AriaProtocol/main-contracts/pull/147>

INFORMATIONAL **I-04.2**

RESOLVED

Total Supply Less Than Minted

CATEGORY Documentation
LOCATION Aria.sol
REVIEW Main Review

— DESCRIPTION

Because the `minted` variable tracks cumulative mints without comparing against `totalSupply`, it is possible for `totalSupply() < minted` after a user burns. This should be clearly documented to avoid confusion.

— RECOMMENDATION

Document that supply can be less than `minted`.

— RESOLUTION

Status: Resolved.

<https://github.com/AriaProtocol/main-contracts/pull/147>

INFORMATIONAL **I-05.2**

ACKNOWLEDGED

Consider Zero Amount Check

CATEGORY	Validation
LOCATION	Aria.sol
REVIEW	Main Review

— DESCRIPTION

The `mint` function does not check if the given amount is > 0 . This could potentially lead to emitting a misleading event.

— RECOMMENDATION

Consider to add a zero amount check.

— RESOLUTION

Status: Acknowledged.

INFORMATIONAL I-03

RESOLVED

Lack Of Zero Address Check On Admin

CATEGORY	Validation
LOCATION	https://github.com/GuardianOrg/main-contracts-team1-1761523410273/blob/1daed28ea7ba4c30d8020e3948a07a3477ba4d3f/contracts/claim/erc20-native/shared/ClaimAdmin.sol#L48
REVIEW	Main Review

DESCRIPTION

The initializer does not validate that `_admin` is non-zero. If initialized with `address(0)`, no one will hold `DEFAULT_ADMIN_ROLE`, which disables all core functionality.

RECOMMENDATION

Add a zero-address check for `_admin` and revert with `ClaimAdmin__ZeroAddress()` if `_admin == address(0)`.

RESOLUTION

Status: Resolved.

<https://github.com/AriaProtocol/main-contracts/pull/166>

INFORMATIONAL **I-04**

RESOLVED

Unused Import

CATEGORY	Superfluous Code
LOCATION	https://github.com/GuardianOrg/main-contracts-team1-1761523143397/blob/1daed28ea7ba4c30d8020e3948a07a3477ba4d3f/contracts/claim/erc20-native/vesting/SnapshotVesting.sol#L4
REVIEW	Main Review

DESCRIPTION

The `IERC20` interface is imported in the `SnapshotVesting.sol` file but not used.

RECOMMENDATION

Consider to remove unused code.

RESOLUTION

Status: Resolved.

<https://github.com/AriaProtocol/main-contracts/pull/167>

INFORMATIONAL I-05

RESOLVED

Allocations Made For Zero Address

CATEGORY	Warning
LOCATION	https://github.com/GuardianOrg/main-contracts-team1-1761523410273/blob/1daed28ea7ba4c30d8020e3948a07a3477ba4d3f/contracts/claim/erc20-native/vesting/CliffReleaseVesting.sol#L68
REVIEW	Main Review

DESCRIPTION

Function `updateAllocations` accepts `wallet = address(0)` in the Allocation struct, which will strand a portion of `totalAllocated`.

RECOMMENDATION

Ensure allocations are appropriately passed by the `DEFAULT_ADMIN_ROLE` or add contract-level validation.

RESOLUTION

Status: Resolved.

<https://github.com/AriaProtocol/main-contracts/pull/168>

INFORMATIONAL I-06

RESOLVED

100% Vesting At Cliff Not Possible

CATEGORY Best Practices

LOCATION <https://github.com/GuardianOrg/main-contracts-team1-1761523410273/blob/1daed28ea7ba4c30d8020e3948a07a3477ba4d3f/contracts/claim/erc20-native/vesting/CliffReleaseVesting.sol#L122>

REVIEW Main Review

DESCRIPTION

Function `_checkCliffReleasePercentage` forbids setting the percentage of the cliff allocation unlock to 100%. If the product requirement includes "full unlock at cliff", it will not be supported.

RECOMMENDATION

Consider if this is expected behavior and clearly document if so. `ERC20NativeClaim` can be used for 100% claims but a merkle root needs to be created.

RESOLUTION

Status: Resolved.

<https://github.com/AriaProtocol/main-contracts/pull/169>

INFORMATIONAL **I-07**

RESOLVED

Potential Overflow On Released Calculation

CATEGORY	Warning
LOCATION	BaseVesting.sol
REVIEW	Main Review

DESCRIPTION

The calculation `uint256 userTotalReleased = (userAmount * totalReleased) / totalAmount;` can potentially overflow when performing `(userAmount * totalReleased)` on large values before being able to reach the division.

RECOMMENDATION

Consider using `mulDiv` or bound allocation amounts appropriately.

RESOLUTION

Status: Resolved.

<https://github.com/AriaProtocol/main-contracts/pull/170>

INFORMATIONAL **I-08**

RESOLVED

Streamlining Storage

CATEGORY	Warning
LOCATION	Global
REVIEW	Main Review

— DESCRIPTION

Both ERC7201 namespaced storage is used and plain storage is used, so care must be taken with upgrades.

— RECOMMENDATION

Consider sticking with just ERC7201 namespaced storage.

— RESOLUTION

Status: Resolved.

<https://github.com/AriaProtocol/main-contracts/pull/148>

LOW **L-01**

RESOLVED

Invalid ClaimAdmin__ZeroAddress Revert

CATEGORY	Best Practices
LOCATION	https://github.com/GuardianOrg/main-contracts-team1-1761523143397/blob/09399fab4bb0bcd76cdf71c667b5c7cd4f1687f/contracts/claim/erc20-native/vesting/BaseVesting.sol#L137
REVIEW	Remediation Review

DESCRIPTION

The function `vestedAmount` reverts when `token_ != $.setup.token` with `ClaimAdmin__ZeroAddress()`, which does not represent the true reason for the error.

RECOMMENDATION

Change the revert name to `BaseVesting__TokenMismatch`.

RESOLUTION

Status: Resolved.

<https://github.com/AriaProtocol/main-contracts/pull/174>

LOW **L-02**

RESOLVED

Arbitrary Token Can Be Deposited

CATEGORY Validation
LOCATION ClaimAdmin.sol
REVIEW Remediation Review

DESCRIPTION

As noted in https://www.notion.so/guardianaudits/L-14-29b8bda5828c810e809ee0c279e9091d?source=copy_link, a `_token` that is not the token being vested can be deposited by users which will credit towards `depositedRewards` and emit a `Deposited` event. This may lead to issues where a user deposits the valid vesting token, the admin changes the vesting token, but users continue to deposit the old vesting token.

RECOMMENDATION

Consider restricting `function deposit(address _token, uint256 _amount)` so that the `_token` must match the `$.setup.token`.

RESOLUTION

Status: Resolved.

<https://github.com/AriaProtocol/main-contracts/pull/174>

INFORMATIONAL **I-01**

RESOLVED

startTime After cliff Is Possible

CATEGORY	Validation
LOCATION	https://github.com/GuardianOrg/main-contracts-team1-1761523143397/blob/09399fab4bb0bcdd76cdf71c667b5c7cd4f1687f/contracts/claim/erc20-native/vesting/BaseVesting.sol#L84-L87
REVIEW	Remediation Review

DESCRIPTION

It is possible to set the `startTime` after the `cliff` with the `setStartTime` function which does not make sense.

RECOMMENDATION

Consider to revert in that case.

RESOLUTION

Status: Resolved.

<https://github.com/AriaProtocol/main-contracts/pull/175>

INFORMATIONAL **I-02**

RESOLVED

PausableUpgradable Not Used

CATEGORY Best Practices
LOCATION ClaimAdmin.sol
REVIEW Remediation Review

DESCRIPTION

Currently the ClaimAdmin uses `Pausable` from solidstate instead of `PausableUpgradable` from OZ which is inconsistent with the other inherited contracts: `AccessControlEnumerableUpgradable` and `UUPSUpgradable`.

RECOMMENDATION

Consider using OZ's `PausableUpgradable` or clearly document this.

RESOLUTION

Status: Resolved.

<https://github.com/AriaProtocol/main-contracts/pull/175>

INFORMATIONAL **I-03**

RESOLVED

Small Amounts Rounds Down To 0

CATEGORY	Warning
LOCATION	BaseVesting.sol
REVIEW	Remediation Review

DESCRIPTION

BaseVesting floors both the stream and per-user share. Consequently, tiny allocations therefore vest to zero and never cross the BaseVesting__ZeroReleased guard, leaving some users unable to claim and producing stranded dust.

RECOMMENDATION

Clearly document this behavior.

RESOLUTION

Status: Resolved.

<https://github.com/AriaProtocol/main-contracts/pull/175>

INFORMATIONAL **I-04**

RESOLVED

Redundant Check

CATEGORY	Best Practices
LOCATION	MerkleWhitelist.sol
REVIEW	Remediation Review

DESCRIPTION

Function `setMerkleRoot` validates that the `_merkleRoot` is not empty bytes with `if (_merkleRoot == bytes32(0)) revert MerkleWhitelist__ZeroMerkleRoot();`. However, `_setMerkleRoot` performs the exact same check: `if (_merkleRoot == bytes32(0)) return;`

RECOMMENDATION

Consider removing the redundant check.

RESOLUTION

Status: Resolved.

<https://github.com/AriaProtocol/main-contracts/pull/175>

INFORMATIONAL I-05

RESOLVED

Invalid Comment For Native Token

CATEGORY Documentation
LOCATION ClaimAdminStorage.sol
REVIEW Remediation Review

DESCRIPTION

"The token to be claimed - `address(0)` for native token, otherwise ERC20." is invalid since the native token is represented by the `NATIVE_SENTINEL_ADDR = 0x00000000000000000000000000000000dEaD`

RECOMMENDATION

Update the comment to appropriately reflect the native token.

RESOLUTION

Status: Resolved.

<https://github.com/AriaProtocol/main-contracts/pull/175>

§ 05

Disclaimer.

This report is not, nor should be considered, an "endorsement" or "disapproval" of any particular project or team. This report is not, nor should be considered, an indication of the economics or value of any "product" or "asset" created by any team or project that contracts Guardian to perform a security assessment. This report does not provide any warranty or guarantee regarding the absolute bug-free nature of the technology analyzed, nor do they provide any indication of the technologies proprietors, business, business model or legal compliance.

This report should not be used in any way to make decisions around investment or involvement with any particular project. This report in no way provides investment advice, nor should be leveraged as investment advice of any sort. This report represents an extensive assessing process intending to help our customers increase the quality of their code while reducing the high level of risk presented by cryptographic tokens and blockchain technology.

Blockchain technology and cryptographic assets present a high level of ongoing risk. Guardian's position is that each company and individual are responsible for their own due diligence and continuous security. Guardian's goal is to help reduce the attack vectors and the high level of variance associated with utilizing new and consistently changing technologies, and in no way claims any guarantee of security or functionality of the technology we agree to analyze.

The assessment services provided by Guardian is subject to dependencies and under continuing development. You agree that your access and/or use, including but not limited to any services, reports, and materials, will be at your sole risk on an as-is, where-is, and as-available basis.

Cryptographic tokens are emergent technologies and carry with them high levels of technical risk and uncertainty. The assessment reports could include false positives, false negatives, and other unpredictable results. The services may access, and depend upon, multiple layers of third-parties.

Notice that smart contracts deployed on the blockchain are not resistant from internal/external exploit. Notice that active smart contract owner privileges constitute an elevated impact to any smart contract's safety and security. Therefore, Guardian does not guarantee the explicit security of the audited smart contract, regardless of the verdict.

§ 06

About Guardian.

Guardian is a smart contract security firm. Every engagement is worked by two independent teams competing on the same scope, each with its own tooling and methods, so no single reviewer's blind spot survives.

\$45B+DIGITAL ASSETS UNDER
REVIEW**200+**

ENGAGEMENTS COMPLETED

500+CRITICAL & HIGH FINDINGS
REPORTED

Reach us at audits@guardianaudits.com.